Multi-Strategy Retrieval with Reciprocal Rank Fusion over Policy-Gated Data

Tom Tang
Edge Vector Foundation

2026

Abstract

A system and method for retrieving information from policy-gated data stores using three independent search strategies — semantic vector search based on cosine similarity over dense embeddings, keyword search based on term-frequency scoring, and structured metadata filtering — unified through Reciprocal Rank Fusion (RRF) to produce a single ranked result list. Each candidate result is independently subject to fold-based access control enforcing four conjunctive layers: trust distance, cryptographic capabilities, lattice-based security labels, and payment gates. Results that fail any access control check are silently removed from each strategy's candidate set before fusion, with no indication to the querier that excluded results exist. The RRF algorithm combines rankings from the three heterogeneous strategies without requiring score normalization, naturally handling variable-sized candidate sets that arise when different numbers of results pass policy checks across different strategies. Query embeddings are generated locally on the querier's device so that the plaintext query is never exposed to the retrieval system. The final fused ranking is presented as a contiguous sequence with no gaps, preventing information leakage through ranking discontinuities that would reveal the existence of policy-filtered results.

1 Introduction

The present invention relates generally to information retrieval systems operating over access-controlled data, and more particularly to methods and systems for combining multiple heterogeneous search strategies through rank fusion algorithms over data stores where each candidate result is independently subject to multi-layer policy enforcement, ensuring that the retrieval process itself does not leak information about the existence or nature of policy-protected data.

2 Background

2.1 The Multi-Strategy Retrieval Problem

Modern information retrieval systems must serve queries that span multiple modalities of relevance. A user searching for "recent cardiovascular risk factors in diabetic patients" requires semantic understanding (the concept of cardiovascular risk, not just the exact phrase), keyword precision (specific medical terms like "HbA1c" or "LDL"), and structured filtering (date ranges, patient cohorts, study types). No single retrieval strategy excels at all three.

Existing approaches address this through:

Semantic vector search. Dense embedding models map documents and queries into a shared vector space where cosine similarity captures semantic relatedness. This excels at synonymy and paraphrase but struggles with rare terms, exact matches, and structured constraints. Scores are continuous values in $[-1, 1]$ with distribution characteristics that vary by embedding model and corpus.

Keyword search. Term-frequency methods such as BM25 score documents by the statistical importance of matching terms. This excels at precise term matching and rare-term retrieval but fails on paraphrases and semantic equivalence. Scores are unbounded positive reals with distribution characteristics that depend on corpus statistics.

Structured metadata filtering. Queries over structured fields (dates, categories, numeric ranges) use exact matching or range predicates. Results are binary (match or no match) and carry no inherent relevance score.

The fundamental challenge is combining these three strategies into a single ranked list. Score normalization across strategies is unreliable because the score distributions are heterogeneous, corpus-dependent, and not directly comparable.

2.2 The Policy Enforcement Problem in Retrieval

In systems where data is subject to access control, retrieval introduces a distinct class of information leakage risks beyond those present in direct data access:

Existence leakage through result counts. If a query returns 47 results to user A and 23 results to user B, user B can infer the existence of 24 documents they are not authorized to see, even though the content is hidden.

Existence leakage through ranking gaps. If results are numbered and user B sees results ranked 1, 2, 5, 8, 9, the gaps at positions 3, 4, 6, 7 reveal that four documents exist that user B cannot access.

Existence leakage through score discontinuities. If relevance scores jump from 0.92 to 0.71 with no results in between, the gap pattern may reveal the approximate number and relevance of filtered documents.

Query exposure. In semantic search, the query must be converted to an embedding vector. If this conversion happens server-side, the system observes the plaintext query, which may itself be sensitive (e.g., a medical professional searching for a specific patient's condition).

2.3 Limitations of Prior Art

Post-hoc filtering. Most access-controlled search systems first retrieve all matching results, then filter by the querier's permissions. This approach leaks information through result counts and requires the retrieval engine to access all documents regardless of the querier's authorization, creating a window where unauthorized data is in memory.

Score-based fusion. Methods such as CombSUM and CombMNZ normalize scores from different strategies into a common range before combining. Normalization requires knowledge of score distributions, which change as policy filtering removes different numbers of results from each strategy. Normalized scores are unreliable when candidate set sizes vary unpredictably.

Learning-to-rank fusion. Machine learning models trained to combine multiple retrieval signals require labeled training data and produce opaque fusion decisions. These models cannot guarantee that the fusion process does not leak information about filtered results through learned score patterns.

Pre-query authorization. Some systems determine the set of authorized documents before retrieval and restrict the search space accordingly. This approach is computationally expensive for large corpora, does not handle the variable-set-size problem inherent in multi-strategy retrieval, and requires materializing the full authorization set.

None of these approaches provide a unified mechanism for combining heterogeneous retrieval strategies over policy-gated data that prevents information leakage through ranking gaps, handles variable candidate set sizes across strategies without score normalization, and keeps the plaintext query private from the retrieval system.

3 Summary

The present invention provides a system and method for multi-strategy retrieval with rank fusion over policy-gated data interfaces. The invention comprises:

1. **Three-strategy independent retrieval over fold-protected data.** Each of three retrieval strategies — semantic vector search, keyword search, and structured metadata filtering — operates independently over data stored in policy-enforcing fold interfaces. Each strategy generates its own candidate set by evaluating relevance within its modality, and each candidate in each set is independently subject to the fold's four-layer conjunctive access control (trust distance, cryptographic capabilities, lattice-based security labels, and payment gates). Candidates that fail any access control check are removed from that strategy's candidate set before any fusion occurs.

2. **Reciprocal Rank Fusion without score normalization.** The system combines the three strategies' ranked candidate sets using Reciprocal Rank Fusion (RRF). For each candidate d appearing in one or more strategy result lists, the fused score is:

$$\mathrm{RRF}(d) = \sum_{s \in S} \frac{1}{k + r_s(d)}$$

   where S is the set of strategies that include d in their post-policy candidate set, $r_s(d)$ is the rank of d in strategy s, and k is a constant (typically 60). This formulation requires only ordinal rankings, not cardinal scores, eliminating the need for score normalization across heterogeneous strategies.

3. **Silent policy exclusion with no information leakage.** Results that fail fold access control are silently removed from each strategy's candidate set before fusion. The RRF algorithm naturally handles the resulting variable-sized candidate sets because it operates on ranks within each strategy's surviving candidates, not on absolute positions. The final fused ranking is presented as a contiguous sequence (ranks 1, 2, 3, ...) with no gaps, no missing positions, and no indication to the querier that any results were excluded. The querier cannot distinguish between "no result existed at this relevance level" and "a result existed but was policy-filtered."

4. **Local query embedding generation.** The querier generates the dense embedding vector for semantic search locally on their own device. Only the embedding vector is transmitted to the retrieval system, never the plaintext query. Because embedding functions are many-to-one mappings from text to vector space, the original query cannot be uniquely reconstructed from the embedding, providing query privacy.

4 Detailed Description

4.1 Definitions

As used throughout this specification and claims, the following terms have the meanings set forth below.

Fold. A *fold* is an interface over a set of fields $\{f_1, \ldots, f_n\}$, where each field f_i carries a value V_i, a security label ℓ_i from a lattice $(\mathcal{L}, \sqsubseteq)$, a trust-distance policy $W_{n_i} R_{m_i}$ specifying separate write threshold n_i and read threshold m_i, and optional cryptographic capability constraints. Raw data is never returned directly; every query passes through a fold that enforces access policies and returns only the authorized projection. If any policy check fails, the fold returns nothing—no data, no error messages, no structural information. As described in co-pending technical disclosure "System and Method for Policy-Enforcing Composable Data Interfaces with Conjunctive Multi-Layer Access Control" by the same inventors.

Trust distance. *Trust distance* $\tau(u, o) \in \mathbb{N}_0$ is a non-negative integer measuring the relational distance between user u and data owner o through a directed graph of trust assignments. The owner has $\tau = 0$. Trust is additive through intermediaries: if $\tau(a, o) = n$ and $\tau(b, a) = m$, then $\tau(b, o) = n + m$. When multiple paths exist, the system uses the shortest path: $\tau(u, o) = \min_{\text{paths}}(\text{sum of distances along path})$. A field is readable if $\tau \leq m$ where m is the field's read threshold.

Security labels. *Security labels* are elements of a partially ordered set (lattice) $(\mathcal{L}, \sqsubseteq)$ following Denning's information flow model. Each field carries a label ℓ. Information may flow from field f_i to f_j only if $\ell_i \sqsubseteq \ell_j$ (equal or higher classification). This prevents declassification of sensitive data through derived fields.

Cryptographic capabilities. *Cryptographic capabilities* $\mathrm{RX}_k(pk)$ and $\mathrm{WX}_k(pk)$ grant read or write access to the holder of public key pk with a bounded quota counter k that decrements on each use and revokes access when $k = 0$. When both trust-distance and capability constraints are present, both must be satisfied.

Payment gates. *Payment gates* require economic consideration as a condition of access. The cost function $C : \mathbb{N}_0 \to \mathbb{R}_{\geq 0}$ maps trust distance to a non-negative payment amount and must be monotonically non-decreasing.

4.2 System Architecture Overview

The system comprises six components (see FIG. 1):

(a) A query processor that receives incoming queries and dispatches them to the three independent retrieval strategies. For semantic search, the query processor receives a pre-computed embedding vector (generated on the querier's device). For keyword search and metadata filtering, the query processor receives structured query parameters.

(b) A semantic search engine that maintains a vector index of document embeddings and computes cosine similarity between the query embedding and all indexed document embeddings to produce a ranked candidate list.

(c) A keyword search engine that maintains an inverted index over document terms and computes BM25 or equivalent term-frequency scores to produce a ranked candidate list.

(d) A metadata filter engine that evaluates structured predicates (date ranges, category membership, numeric thresholds) against document metadata to produce a candidate set ordered by a configurable sort criterion.

(e) A policy enforcement layer that evaluates each candidate from each strategy against the fold's four-layer conjunctive access control, removing candidates that fail any check and producing

filtered candidate sets for fusion.

(f) A rank fusion engine that applies Reciprocal Rank Fusion across the filtered candidate sets from all three strategies, producing a single contiguous ranked result list.

4.3 Retrieval Strategy Details

4.3.1 Semantic Vector Search

The semantic search strategy operates on dense embeddings in a vector space $\mathbb{R}^n$ where n is the embedding dimensionality. Each document d_i in the data store has a corresponding embedding vector $\mathbf{v}_i$ computed at ingestion time. The query embedding $\mathbf{q}$ is computed on the querier's device using the same embedding model.

The relevance score for each document is the cosine similarity:

$$\mathrm{sim}(d_i) = \frac{\mathbf{q} \cdot \mathbf{v}_i}{\|\mathbf{q}\| \, \|\mathbf{v}_i\|}$$

Documents are ranked by descending similarity score. The strategy returns the top N_{sem} candidates as an ordered list $L_{\mathrm{sem}} = [(d_{(1)}, r_1), (d_{(2)}, r_2), \ldots]$ where $r_j = j$ is the rank.

4.3.2 Keyword Search

The keyword search strategy operates on an inverted index mapping terms to document identifiers and positional information. Given a set of query terms $\{t_1, \ldots, t_m\}$, the relevance score for each document uses BM25 or an equivalent term-frequency model:

$$\mathrm{BM25}(d_i) = \sum_{j=1}^{m} \mathrm{IDF}(t_j) \cdot \frac{f(t_j, d_i) \cdot (k_1 + 1)}{f(t_j, d_i) + k_1 \cdot \left(1 - b + b \cdot \frac{|d_i|}{\mathrm{avgdl}}\right)}$$

where $f(t_j, d_i)$ is the frequency of term t_j in document d_i, $|d_i|$ is the document length, avgdl is the average document length, and k_1, b are tuning parameters. Documents are ranked by descending BM25 score and the top N_{kw} candidates form the ordered list L_{kw}.

4.3.3 Structured Metadata Filtering

The metadata filter strategy evaluates structured predicates against document metadata fields. A metadata query M is a conjunction of predicates:

$$M = p_1 \wedge p_2 \wedge \cdots \wedge p_l$$

where each p_j is a predicate over a metadata field (e.g., date $\geq$ 2025-01-01, category $\in \{\mathrm{cardiology}, \mathrm{endocrinology}\}$, citation_count > 50). A document satisfies M if and only if it satisfies all l predicates. Matching documents are ordered by a configurable sort criterion (e.g., recency, alphabetical, numeric field value) to produce the ordered list L_{meta}.

4.4 Policy Enforcement on Candidate Sets

Each candidate in each strategy's result list is independently evaluated against the fold's four-layer conjunctive access control under the querier's access context $\mathcal{C} = (u, \tau(\cdot,\cdot), K)$, where u identifies the querier, $\tau(u, o)$ is the function computing the querier's trust distance from a data owner o, and K is the set of public keys the querier holds.

For each candidate d_i in a strategy's ranked list, the system evaluates:

(i) **Trust distance**: whether $\tau(u, o_i) \leq m_i$, where o_i is the owner of d_i and m_i is the read threshold of the fold containing d_i.

(ii) **Cryptographic capabilities**: whether the querier holds a valid capability with positive remaining quota for reading d_i.

(iii) **Security labels**: whether the label flow from d_i's fold to the query result satisfies the lattice constraint $\ell_i \sqsubseteq \ell_{\text{result}}$.

(iv) **Payment**: whether the payment condition $\mathcal{P}(u, F_i)$ is satisfied for the fold F_i containing d_i.

All four checks are conjunctive: if any check fails, the candidate is removed from that strategy's result list. The removal is silent — no placeholder, no gap marker, no indication of any kind is recorded in the output. The remaining candidates are re-ranked within the strategy to form a contiguous sequence:

$$L'_s = \text{filter}(L_s, \mathcal{C}) = [(d'_{(1)}, 1), (d'_{(2)}, 2), \ldots, (d'_{(|L'_s|)}, |L'_s|)]$$

where $|L'_s| \leq |L_s|$ and the relative order of surviving candidates is preserved from the original ranking.

4.5 Reciprocal Rank Fusion

The RRF algorithm combines the filtered, re-ranked candidate sets from all three strategies. Let $S = \{\text{sem}, \text{kw}, \text{meta}\}$ be the set of strategies. For each unique candidate d appearing in any filtered list L'_s, the fused score is:

$$\text{RRF}(d) = \sum_{s \in S(d)} \frac{1}{k + r'_s(d)}$$

where $S(d) \subseteq S$ is the set of strategies whose filtered list contains d, $r'_s(d)$ is the rank of d in the filtered list L'_s, and k is a smoothing constant (typically $k = 60$).

The key properties of RRF that make it suitable for policy-gated retrieval are:

(a) Rank-based, not score-based. RRF uses only ordinal rankings, not cardinal scores. This eliminates the need for score normalization across heterogeneous strategies whose score distributions are incomparable (cosine similarity in $[-1, 1]$ versus unbounded BM25 scores versus binary metadata match).

(b) Tolerant of variable set sizes. After policy filtering, different strategies may have vastly different numbers of surviving candidates. RRF naturally handles this because a candidate that appears in only one strategy's list still receives a fused score from that strategy alone, while a candidate appearing in multiple lists receives additive contributions from each. No adjustment is needed for missing entries.

(c) Monotonic in rank. Higher-ranked candidates in any strategy receive higher RRF contributions. The fused ranking preserves the relative importance signal from each strategy without amplifying or suppressing any individual strategy's contribution disproportionately.

(d) Bounded contribution per strategy. Each strategy contributes at most $\frac{1}{k+1}$ to a candidate's fused score (for a rank-1 result). This prevents any single strategy from dominating the fused ranking.

The final output is the set of all unique candidates sorted by descending $\text{RRF}(d)$ score, assigned contiguous ranks $1, 2, 3, \ldots$:

$$R_{\text{final}} = [(d_{(1)}, 1), (d_{(2)}, 2), \ldots, (d_{(|R|)}, |R|)]$$

4.6 Prevention of Information Leakage

The system prevents information leakage through multiple mechanisms:

4.6.1 Contiguous Ranking with No Gaps

If the unfiltered candidate sets from the three strategies collectively contain results at (hypothetical) positions 1 through 20, and policy enforcement removes candidates at positions 3, 7, and 12, the querier sees a contiguous ranking of 17 results numbered 1 through 17. The querier cannot determine:

- That any results were removed.
- How many results were removed.
- At which positions removed results would have appeared.
- Which strategy or strategies contributed the removed results.

This is achieved by re-ranking within each strategy after policy filtering (Section 3) and producing a contiguous fused ranking (Section 4).

4.6.2 No Score Exposure

The system does not expose raw relevance scores, BM25 scores, cosine similarities, or RRF fused scores to the querier. Only the contiguous rank ordering is returned. This prevents the querier from inferring the existence of filtered results through score distribution analysis (e.g., detecting an unusually large gap between consecutive scores that would suggest a missing result).

4.6.3 Query Privacy Through Local Embedding

The querier generates the semantic search embedding vector on their own device using a locally-available embedding model. The system receives only the dense vector $\mathbf{q} \in \mathbb{R}^n$, never the plaintext query string. Because the embedding function $E : \Sigma^* \to \mathbb{R}^n$ maps from an infinite set of possible text strings to a finite-dimensional vector space, it is a many-to-one function:

$$|E^{-1}(\mathbf{q})| = \infty \quad \text{for all } \mathbf{q} \in \text{range}(E)$$

The system cannot uniquely reconstruct the original query from the embedding vector. While approximate inversion attacks exist, they recover only semantically similar text, not the exact query, providing meaningful query privacy.

4.6.4 Independent Strategy Isolation

Each retrieval strategy operates independently and does not share intermediate results, candidate sets, or filtering decisions with other strategies. A candidate that is policy-filtered from the semantic search list but present in the keyword search list is included in the fused ranking based solely on its keyword search rank. The fusion process does not reveal to any strategy which candidates were filtered from other strategies.

4.7 Handling of Multi-Owner Data

In a multi-tenant data store where different documents are owned by different users with different fold policies, the retrieval system evaluates each candidate's fold policy independently. A single query may encounter candidates from folds with different trust distance thresholds, different capability requirements, different security labels, and different payment conditions.

The policy enforcement layer evaluates each candidate against its own fold's policies, not against a single global policy. This means:

- Candidate d_1 from fold F_A (with read threshold $m = 3$) passes for a querier at trust distance $\tau(u, o_A) = 2$.
- Candidate d_2 from fold F_B (with read threshold $m = 1$) fails for the same querier.
- Candidate d_3 from fold F_C (with payment gate requiring \$0.01) passes if the querier has paid.

The querier sees d_1 and d_3 in their results with no indication that d_2 exists.

4.8 Embedding Model Consistency

For semantic search to produce meaningful results, the embedding model used to generate query embeddings on the querier's device must be compatible with the model used to generate document embeddings at ingestion time. The system addresses this through:

(a) Model versioning. Each vector index is associated with a specific embedding model version identifier. The querier's client application retrieves the model identifier and uses the corresponding model for local embedding generation.

(b) Model distribution. The embedding model weights are distributed to querier devices, enabling local inference. The model itself is not secret; only the query text is private.

(c) Version migration. When the embedding model is updated, document embeddings are recomputed. During migration, the system maintains both old and new indexes to support queriers that have not yet updated their local model.

4.9 RRF Parameter Selection

The smoothing constant k in the RRF formula controls the relative influence of rank position on the fused score. For a candidate at rank r, the contribution is $\frac{1}{k+r}$.

- A small k (e.g., $k = 1$) amplifies the difference between top-ranked and lower-ranked candidates: a rank-1 candidate contributes $\frac{1}{2}$ while a rank-10 candidate contributes $\frac{1}{11}$, a 4.5× ratio.
- A large k (e.g., $k = 60$) compresses rank differences: a rank-1 candidate contributes $\frac{1}{61}$ while a rank-10 candidate contributes $\frac{1}{70}$, only a 1.15× ratio.

The system uses $k = 60$ as the default, which provides robust fusion that is not overly sensitive to small rank differences within any single strategy. The parameter k is configurable per deployment but is not exposed to queriers, preventing queriers from manipulating the fusion behavior.

4.10 Audit and Logging

Every retrieval operation is recorded in the audit service:

- The query embedding vector (not the plaintext query, which the system never receives).
- The keyword search terms and metadata predicates.
- The number of candidates generated by each strategy (before and after policy filtering).
- The identifiers of policy-filtered candidates (recorded for the data owner's audit, not exposed to the querier).
- The final fused ranking returned to the querier.
- The access context $\mathcal{C}$ under which the query was evaluated.

The audit log enables data owners to observe how their data is being accessed and how often their fold policies are preventing disclosure, without exposing this information to queriers.

5 Figures

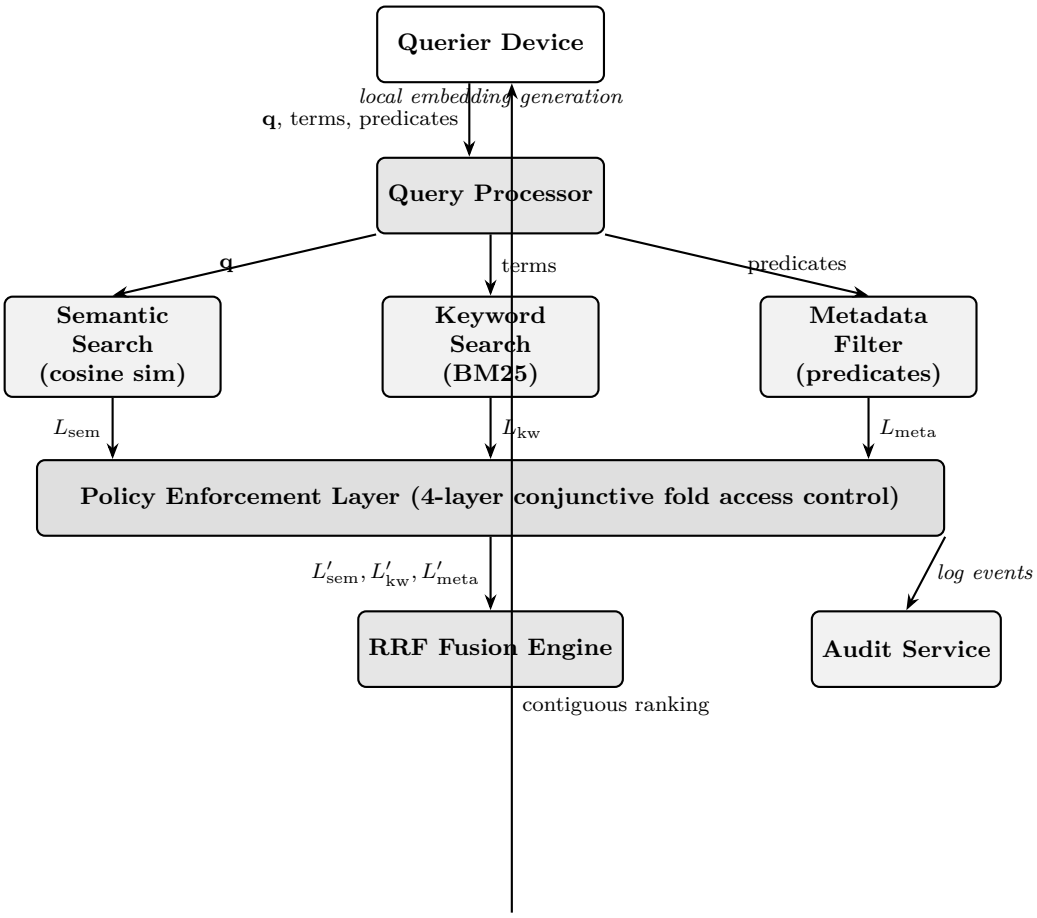


Figure 1: **System Architecture.** Six components: querier device (local embedding), query processor, three independent retrieval engines, policy enforcement layer, RRF fusion engine, and audit service. The plaintext query never leaves the querier's device. Each strategy's candidates pass independently through fold-based access control before fusion.

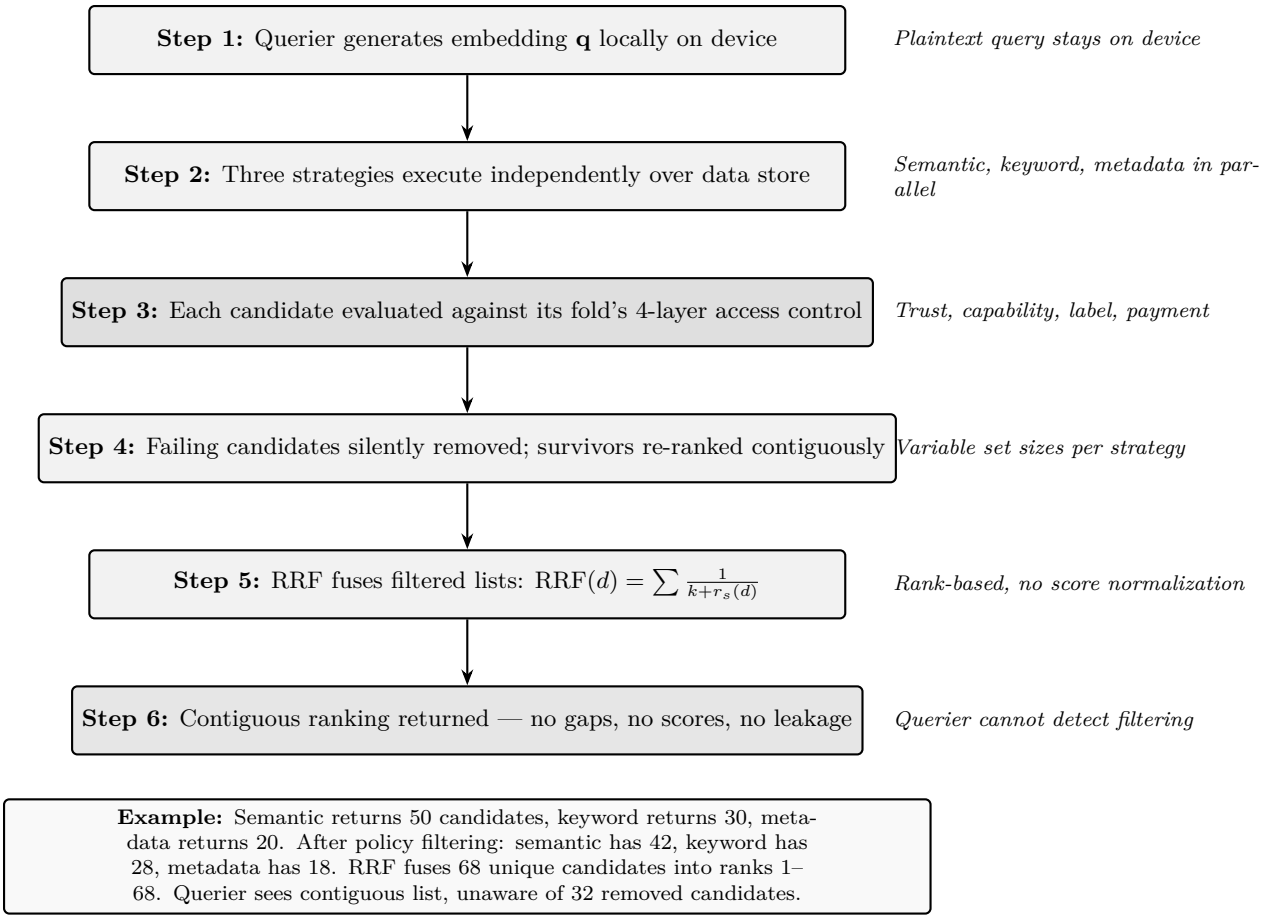
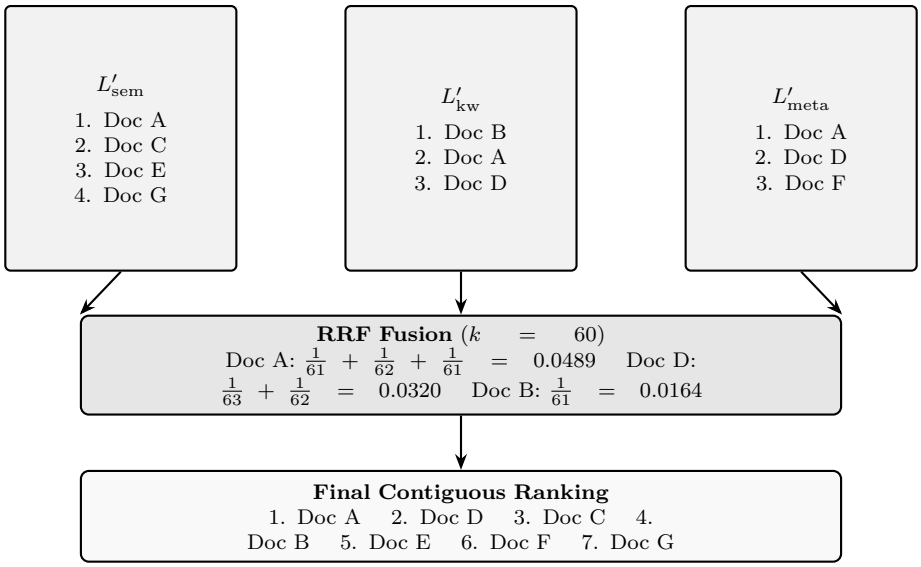


Figure 2: **Query Pipeline Flow.** The six-step pipeline from local embedding generation through three-strategy retrieval, per-candidate policy enforcement, silent exclusion, RRF fusion, and contiguous result delivery. At no point does the querier observe evidence of policy-filtered results.



Candidates appearing in multiple strategies receive additive RRF contributions. Variable list sizes handled naturally. No score normalization required.

Figure 3: **RRF Fusion Detail.** Three filtered candidate lists of different sizes are combined through Reciprocal Rank Fusion. Each strategy contributes $\frac{1}{k+\text{rank}}$ per candidate. Candidates in multiple lists accumulate contributions, producing a single contiguous ranking.